Answer 1: INITIALISATION, TRACKING, AND MAPPING IN SLAM (25 Marks)

1. Introduction to SLAM

Concept and Importance:
Simultaneous Localization and Mapping (SLAM) is a foundational computational problem in robotics and autonomous systems. As detailed in the provided lecture materials (Steps involved in...SLAM.pptx), it addresses a "chicken-and-egg" problem: an autonomous agent, placed in an unknown environment without a priori knowledge or external aids (like GPS), must concurrently build a consistent map of that environment and simultaneously determine its own pose (position and orientation) within that map.
The importance of SLAM is paramount; it is the core enabler of true autonomy. Without SLAM, a robot is merely following pre-programmed paths in a known environment. With SLAM, it gains the ability to navigate, explore, and interact with unknown, dynamic spaces, forming the basis for autonomous navigation, path planning, and environmental interaction.

Historical Evolution and Overview:
Early SLAM research (1990s) was dominated by probabilistic filter-based approaches, most notably EKF-SLAM (Extended Kalman Filter). This method, often using 2D LiDAR, treated the robot's pose and all map landmarks as a single, large state vector. While foundational, its O(N^2) computational complexity (where N is the number of landmarks) made it intractable for large-scale maps.
This limitation led to solutions like **FastSLAM** (c. 2000s), which used a Rao-Blackwellized particle filter to improve scalability. However, the modern paradigm, particularly for high-dimensional Visual SLAM (V-SLAM), has shifted decisively to **optimization-based** methods. These **Graph-based** approaches formulate SLAM as a non-linear least-squares optimization problem, which is far more scalable and efficient.

2. Mathematical Formulation of SLAM

The SLAM problem is fundamentally one of state estimation. We seek to compute the posterior probability distribution for the robot's state and the map, given all sensor measurements and control inputs.

State Estimation Model:
Let:

- x_k = Robot state (pose) at time k.
- m = The map, represented as a set of all landmark positions {m_i}.
- u_k = Control input or odometry (e.g., IMU, wheel encoders) at time k.
- z_k = Observation (measurement) of landmarks at time k.
- X_{0:k} = {x_0, ..., x_k} = History of robot poses.
- Z_{1:k} = {z_1, ..., z_k} = History of all observations.
- U_{1:k} = {u_1, ..., u_k} = History of all control inputs.

The full SLAM problem is to compute the posterior over the entire path and map:
P(X_{0:k}, m | Z_{1:k}, U_{1:k})
The online SLAM problem (more common for real-time robotics) is to compute the posterior for the current pose and map:
P(x_k, m | Z_{1:k}, U_{1:k})
Probabilistic Representation (Bayesian Framework):
This posterior is estimated recursively using a two-step Bayesian filter:

1. Prediction (Motion Model): The state is propagated forward using the motion model P(x_k | x_k-1, u_k). This predicts the robot's new pose based on odometry, incorporating process noise.
   P(x_k, m | Z_{1:k-1}, U_{1:k}) = ∫ P(x_k | x_k-1, u_k) P(x_k-1, m | Z_{1:k-1}, U_{1:k-1}) dx_{k-1}
2. Update (Observation Model): The predicted state is corrected using the current observation z_k via the observation model P(z_k | x_k, m). This model defines the likelihood of seeing measurement z_k from pose x_k given the map m.
   P(x_k, m | Z_{1:k}, U_{1:k}) ∝ P(z_k | x_k, m) * P(x_k, m | Z_{1:k-1}, U_{1:k})

Solving this full posterior is intractable due to the high dimensionality and non-linear nature of the models. EKF-SLAM linearizes the models, while Graph-SLAM assumes Gaussian noise and solves the corresponding least-squares optimization problem, which is an approximation of this probabilistic framework.

3. Phases of SLAM

As emphasized in Steps involved in...SLAM.pptx, modern SLAM systems are structured as a **Frontend** (real-time processing) and a **Backend** (batch optimization).

<Flowchart of complete SLAM process>

Initialization (Bootstrapping):
This is the critical frontend process of building the first piece of the map to establish a world coordinate system.

- **Keyframe Selection:** The system selects an initial "Keyframe" to serve as the origin.
- **Feature Extraction:** Salient features (e.g., ORB, SIFT) are extracted.

- **Reference Frame Definition:** The system waits for a second frame with sufficient motion (parallax). For **monocular SLAM**, this is non-trivial. It requires computing a Fundamental Matrix or Homography between the two views to triangulate the 3D positions of initial features. This triangulation establishes the initial map *up to an unknown scale*, as noted in Monocular_Stereo_RGBD...pptx. For Stereo or RGB-D, this is simpler as metric depth is available immediately.

Tracking (Frontend):
This is the real-time, frame-by-frame process of estimating the sensor's pose relative to the existing map.

- **Pose Estimation:** For each new frame, features are matched to the 3D map points (landmarks) created in previous steps. The camera's 6-DoF pose is then efficiently computed by solving a **Perspective-n-Point (PnP)** problem, which finds the pose that minimizes the **reprojection error**. In *Direct* methods (like LSD-SLAM), this step minimizes the *photometric error* (pixel intensity differences) instead (Direct_vs_Indirect...pptx).
- **Feature Association (Data Association):** This is the crucial sub-problem of correctly matching observed features to known landmarks. As described in SLAM by PSanthanam.ppt, a common method is **Nearest Neighbour**, where an extracted feature is matched to the closest landmark in the database. This match is then passed through a **validation gate** (e.g., using Mahalanobis distance or RANSAC) to reject outliers, which are critical non-Gaussian errors that can corrupt the estimate.

<Trajectory tracking and pose estimation example>

Mapping (Backend):
This is the backend process, running in a separate thread, that refines the map and poses for global consistency.

- **Map Representation:**
  - **Sparse:** A collection of 3D feature points. Efficient and robust. (e.g., **ORB-SLAM**).
  - **Semi-Dense:** 3D points for pixels with high image gradients. (e.g., **LSD-SLAM**).
  - **Dense:** A full 3D model (e.g., KinectFusion).
- **Loop Closure:** This is the critical task of recognizing a previously visited place (SLAM by PSanthanam.ppt, Steps involved in...SLAM.pptx). This is a place recognition problem, often solved using a Bag-of-Words (BoW) approach on feature descriptors.
- **Map Optimization:** When a loop is detected, it creates a powerful constraint.
  1. **Pose Graph Optimization:** The backend builds a **Pose Graph** where nodes are Keyframe poses and edges are constraints (relative poses from odometry or loop closures). The new loop closure adds an edge. A non-linear least-squares optimizer (e.g., g2o, Ceres) then adjusts *all* poses in the graph to minimize the total error, correcting the accumulated **drift** (Steps involved in...SLAM.pptx).
  2. **Bundle Adjustment (BA):** The "gold standard" of V-SLAM. BA is a joint, non-linear optimization of *all* Keyframe poses and *all* 3D landmark positions to minimize the global reprojection error. It is computationally expensive and typically run by the

backend after loop closure or on a local window of Keyframes.

<Visualization of keyframe-based mapping>

4. Algorithms and Techniques Used

- **EKF-SLAM:** (Filter-based) The original probabilistic approach. Fuses robot pose and all landmarks into one state vector. Does not scale due to the O(N^2) update of the covariance matrix.
- **FastSLAM:** (Filter-based) A Rao-Blackwellized particle filter. Each particle maintains a separate, low-dimensional map. More scalable than EKF-SLAM but suffers from particle depletion and is less accurate than optimization methods.
- **Graph-based SLAM:** (Optimization-based) The modern standard. Formulates SLAM as a pose graph optimization, a sparse non-linear least-squares problem. Highly scalable and accurate, especially with loop closures.
- **ORB-SLAM:** (Indirect/Feature-based) As detailed in Direct_vs_Indirect...pptx and Monocular_Stereo_RGBD...pptx, this is a highly robust V-SLAM system. It is **indirect**, using ORB features, and minimizes **reprojection error**. Its three-thread (Tracking, Local Mapping, Loop Closing) architecture allows for real-time performance and strong robustness.
- **LSD-SLAM:** (Direct-based) A key example of a **direct** method from Direct_vs_Indirect...pptx. It operates directly on pixel intensities, minimizing **photometric error**. It produces semi-dense maps, which are more detailed, but it is more sensitive to photometric changes (e.g., lighting, auto-exposure) than feature-based methods.

Comparative Discussion:
<Table: Comparison of different SLAM algorithms>

Algorithm	**Method Type**	**Map Representation**	**Scalability**	**Key Characteristic**
EKF-SLAM	Filtering (EKF)	Sparse (Landmarks)	Poor (O(N^2))	High-correlation state, computationally expensive.
FastSLAM	Filtering (Particle)	Sparse (per-particle)	Good	Rao-Blackwellized. Solves non-linear

				motion, but particle-heavy.
Graph-SLAM	Optimization (Least-Squares)	Pose Graph	Excellent	Backend optimization. Handles loop closure very well.
ORB-SLAM	Optimization (Indirect/Feature)	Sparse (Keypoints)	Excellent	Real-time, robust to motion, strong loop closure. Minimizes reprojection error.
LSD-SLAM	Optimization (Direct/Intensity)	Semi-Dense (Pixels)	Good	Dense map, high accuracy, but sensitive to lighting. Minimizes photometric error.

5. Error Sources and Uncertainty Management

- **Noise in Sensors:** All sensor measurements (u_k and z_k) are corrupted by noise, typically modeled as zero-mean Gaussian. This is the fundamental source of uncertainty that the probabilistic framework (filters or least-squares) is designed to manage.
- **Data Association Errors:** As noted in SLAM by PSanthanam.ppt, this is the critical, non-Gaussian error of incorrectly matching an observation to the wrong landmark. This can cause the entire estimate to diverge. It is managed by:
  - **Robust Algorithms:** Using RANSAC (Random Sample Consensus) during initialization or PnP tracking.
  - **Validation Gates:** Rejecting matches that are statistically "too far" from their predicted location (e.g., using Mahalanobis distance).
- **Drift Compensation:** This refers to the *management* of small, incremental errors from

sensor noise and imperfect modeling that accumulate over time, causing the estimated trajectory to "drift" from the true path. The primary compensation mechanism is **Loop Closure**, which provides a large-scale constraint to the backend optimizer, allowing the accumulated error to be corrected.

6. Applications in Automation and Robotics

- **Mobile Robot Navigation:** Autonomous Ground Vehicles (AGVs) in warehouses and service robots (as mentioned in Monocular_Stereo_RGBD...pptx) use 2D LiDAR or RGB-D SLAM to map and navigate floors.
- **Autonomous Drones:** V-SLAM or Visual-Inertial SLAM (VINS, as in ORB-SLAM3) is essential for drones to navigate in 3D, GPS-denied environments.
- **Robotic Manipulators:** In "mobile manipulation," SLAM is used to localize the mobile base, allowing the arm to interact with objects in an unstructured environment.
- **Augmented/Virtual Reality (AR/VR):** SLAM provides the "inside-out" tracking that allows a headset to understand its position and map the room in real-time.

7. Conclusion

The SLAM pipeline, comprising the frontend (Initialization, Tracking) and backend (Mapping, Loop Closure, Optimization), is a complex but solved problem that forms the bedrock of modern autonomous systems. As shown in the provided materials, the field has evolved from filter-based methods to highly scalable and robust optimization-based frameworks (Graph-SLAM). The choice between algorithms, such as Direct (LSD-SLAM) and Indirect (ORB-SLAM), represents a key engineering trade-off between map density, computational efficiency, and robustness to environmental conditions.

References:

- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
- Durrant-Whyte, H., & Bailey, T. (2006). Simultaneous Localization and Mapping (SLAM): Part I. *IEEE Robotics & Automation Magazine*.
- Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: A Versatile and Accurate Monocular SLAM System. *IEEE Transactions on Robotics*.
- Engel, J., Schöps, T., & Cremers, D. (2014). LSD-SLAM: Large-Scale Direct Monocular SLAM. *ECCV*.

Answer 2: MONOCULAR, STEREO, AND RGB-D SENSORS (25 Marks)

1. Introduction

Role of Vision Sensors:
Vision sensors are fundamental components in computer vision and robotics, providing rich, high-bandwidth data about the environment. As discussed in Monocular_Stereo_RGBD...pptx, they are the primary sensor for Visual SLAM and are critical for tasks including object detection, recognition, segmentation, visual servoing, and autonomous navigation.
Overview of Depth Perception Methods:
A core challenge in computer vision is recovering the 3D structure of the world from 2D images. A single 2D image is an ambiguous projection of the 3D world; a small, nearby object can project an image identical to a large, distant object. Monocular, Stereo, and RGB-D systems represent three distinct, widely-used approaches to resolving this ambiguity and perceiving depth.

2. Monocular Vision System

- Working Principle: A monocular system uses a single camera. It operates based on the pinhole camera model, which projects 3D world points (X, Y, Z) onto a 2D image plane (u, v).
  <Diagram of Monocular camera setup>
- **Depth Estimation:** A single static image **cannot** recover absolute (metric) depth. 3D structure can only be recovered *up to an unknown scale*. To estimate 3D geometry, the camera must be in motion, a process known as **Structure from Motion (SfM)**. By tracking features across multiple frames, the system observes their **parallax** (apparent motion) and can triangulate their 3D positions, but the entire reconstruction's scale is arbitrary.
- **Limitations in SLAM:**
  1. **Scale Ambiguity:** As highlighted in Monocular_Stereo_RGBD...pptx, if a monocular SLAM system moves 10 "units," it does not know if that is 10 cm or 10 meters. This is problematic for robotic control.
  2. **Initialization:** Requires a dedicated "bootstrapping" procedure, needing two frames with sufficient motion to triangulate the first set of 3D points.
  3. **Scale Drift:** Small pose estimation errors accumulate, causing the estimated scale of the map to "drift" over time.

- **Applications:** Ideal where cost, size, power, and weight are primary constraints. (e.g., micro-drones, mobile phone AR, lightweight robots).

3. Stereo Vision System

- Working Principle: A stereo system uses two cameras mounted with a fixed, known separation called the baseline (b). It captures two images simultaneously, mimicking human binocular vision.
  <Stereo vision setup showing baseline and disparity>
- **Epipolar Geometry:** This is the geometry of stereo vision (Szeliski, Ch. 11). A 3D point P projects to p_L in the left image and p_R in the right. Epipolar geometry provides a powerful constraint: the search for the corresponding point p_R is restricted to a 1D line in the right image, known as the **epipolar line**. In a calibrated and **rectified** system, this line is horizontal.
- **Correspondence Problem:** This is the core challenge. For each pixel in the left image, the algorithm must find its corresponding partner in the right image, typically by comparing small image patches (e.g., using Sum of Absolute Differences - SAD). This process fails in texture-less regions (e.g., white walls).
- **Disparity Map Computation:** The result of solving correspondence is the **disparity (d)**, which is the difference in the horizontal pixel coordinates: d = u_L - u_R.
- Depth Computation: Given a calibrated and rectified stereo rig, depth Z is recovered directly via triangulation:
  Z = (b * f) / d
  Where f is the camera's focal length. Depth is inversely proportional to disparity. This provides metric scale from the outset (Monocular_Stereo_RGBD...pptx).
- **Accuracy Factors:** Accuracy (ΔZ) degrades quadratically with distance (Z^2). It is improved by a *larger baseline (b)* (which increases d for distant objects) and a *longer focal length (f)*.

4. RGB-D Sensor System

- Principle of Operation: An RGB-D (Red, Green, Blue + Depth) sensor is an active sensor. It pairs a standard RGB camera with a dedicated depth-sensing unit that projects its own illumination (typically infrared).
  <Working principle of RGB-D sensor>
  There are two primary technologies:
  1. **Structured Light (SL):** Projects a known, complex pattern of infrared light onto the scene. An IR camera captures the *deformation* of this pattern, and triangulation is

used to compute depth for each pixel. *Examples: Microsoft Kinect v1, Intel RealSense SR300 series*.

2. **Time-of-Flight (ToF):** Emits pulses of infrared light and measures the precise *round-trip time* for the light to travel to an object, reflect, and return. Since the speed of light is constant, time directly yields distance. *Examples: Microsoft Kinect v2 (Azure Kinect), Intel RealSense L515*.

- **Depth Computation:** The sensor's onboard hardware/firmware computes the depth map directly, providing it as a 2.5D image (where each pixel's value is its distance) pre-registered with the RGB image. This significantly reduces the computational load on the robot's processor.
- **Limitations:** Being active IR sensors, they are easily "blinded" by sunlight (which is full of IR), making them unsuitable for most outdoor applications. They also fail on transparent (glass) or highly specular (mirror) surfaces and have a limited effective range (e.Code, 0.5m - 5m).

5. Comparative Analysis

This table is synthesized from the comparative slides in Monocular_Stereo_RGBD...pptx.
<Table comparing Monocular vs Stereo vs RGB-D sensors>

Parameter	Monocular Vision	Stereo Vision	RGB-D Sensor
Depth Principle	Structure from Motion (SfM)	Triangulation (Binocular Parallax)	Active (Structured Light or ToF)
Depth Output	Relative (Scale-ambiguous)	Metric (via Z=bf/d)	Metric (Direct hardware measurement)
Depth Accuracy	Low, prone to scale drift	Medium (degrades quadratically with distance)	High (within optimal range)
Computational Cost	Low (image) / High (SfM/SLAM)	High (Stereo correspondence)	Low (Depth is provided by hardware)
Sensor Cost	Very Low	Low to Medium	Medium

Robustness (Lighting)	High (Passive)	High (Passive, but needs light)	**Low** (Active IR fails in sunlight)
Robustness (Texture)	High (features) / Low (direct)	**Low** (Fails in texture-less areas)	**High** (Works without texture)
Data Output Format	2D Image (RGB)	2x 2D Images (RGB)	2D Image + 2.5D Depth Map
Suitability for SLAM	Drones, AR (low weight). (e.g., ORB-SLAM Mono)	Autonomous Vehicles, Outdoor (metric, passive). (e.g., ORB-SLAM Stereo)	Indoor Robots (dense, low-compute). (e.g., ORB-SLAM RGB-D)

6. Calibration and Error Handling

Camera Calibration:
All vision sensors require calibration to find their parameters, correcting for the geometric imperfections of the lens and sensor alignment (Szeliski, Ch. 6; Gonzales & Woods, Ch. 4).

- **Zhang's Method:** A common technique using a planar checkerboard pattern. By capturing the pattern from multiple viewpoints, an optimization algorithm can solve for the camera's parameters.
- **Intrinsic Parameters:** These are internal to the camera:
  - **Focal Length** (f_x, f_y)
  - **Principal Point** (c_x, c_y): The pixel coordinate of the optical axis.
- **Distortion Correction:** Calibration also solves for **lens distortion coefficients** (e.g., radial k_1, k_2 and tangential p_1, p_2). These model physical imperfections like "barrel" or "pincushion" distortion. All subsequent processing is done on the *undistorted* or *rectified* image.
- **Extrinsic Parameters:** This is the 6-DoF rigid-body transformation [R | t] (Rotation and Translation) *between* coordinate frames.
  - For **Stereo:** This is the precise pose of the right camera relative to the left.
  - For **RGB-D:** This is the precise pose of the depth sensor relative to the RGB camera.
- **Synchronization Issues:** A critical error source for Stereo and RGB-D. The image-capturing (left/right or RGB/depth) must be **hardware-synchronized**. If the frames are captured at even slightly different times (Δt), a moving object will be in

different positions, leading to catastrophic errors in depth computation.

7. Applications in Robotics

- **Mobile Robot Navigation:** As per Monocular_Stereo_RGBD...pptx, **RGB-D** is ideal for indoor service robots for dense obstacle avoidance. **Stereo** is the standard for outdoor autonomous vehicles.
- **Obstacle Avoidance:** Stereo and RGB-D sensors provide a direct 3D point cloud of the environment, which can be projected into a 2D "costmap" for a mobile robot to plan paths.
- **Visual Servoing and Manipulation:** Guiding a robotic arm to grasp an object. **RGB-D** is extremely powerful here, as it provides the full 3D pose and shape of the target object, enabling sophisticated grasp planning. **Monocular** visual servoing is also possible but more complex.

8. Conclusion

The choice between monocular, stereo, and RGB-D sensors is a critical, application-dependent trade-off in robotic system design.

- **Monocular** is the cheapest, smallest, and most versatile (no range/lighting limits), but burdens the software with the difficult problem of scale estimation and drift.
- **Stereo** is a robust, passive solution that provides metric depth, making it the standard for autonomous vehicles and outdoor robotics, but it fails in texture-poor environments.
- **RGB-D** sensors provide dense, accurate, metric depth with low computational load, revolutionizing indoor robotics, but their active-nature limits their range and renders them unusable outdoors.

As noted in Monocular_Stereo_RGBD...pptx, modern high-performance systems (like ORB-SLAM3) mitigate the weaknesses of a single sensor by using **sensor fusion**, most commonly Visual-Inertial SLAM (VINS), which pairs a camera (monocular or stereo) with an IMU for high-frequency, robust pose estimation.

References:

- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
- Gonzales, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.).
- Zhang, Z. (2000). A Flexible New Technique for Camera Calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

- Mur-Artal, R., & Tardós, J. D. (2017). ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras. *IEEE Transactions on Robotics*.